

WINE

Техническая документация backend · FastAPI + YOLO(ONNX) + DINOv2 + FAISS + PostgreSQL

WINE

Техническая документация backend

FastAPI + YOLO (ONNX Runtime) + DINOv2 with Registers + FAISS + PostgreSQL

Назначение документа

Документ описывает фактическую backend-реализацию по предоставленному архиву репозитория Wine_back: архитектуру, ML-пайплайн распознавания вина, сетевые эндпоинты, работу с внешним сайтом-источником данных, кэширование, конфигурацию, сборку и эксплуатационные замечания. Документ предназначен для разработчика, которому необходимо быстро понять кодовую базу и продолжить её развитие.

Параметр	Значение
Проект	Wine_back
Модель приложения	Единый Python-сервис (монолит)
Framework	FastAPI + Uvicorn
Компьютерное зрение	YOLO (ONNX Runtime) — детекция этикетки
Embedding-модель	DINOv2 with Registers (facebook/dinov2-with-registers-small)
Поиск похожих	FAISS (IndexFlatIP, cosine similarity)
Источник данных о винах	Парсинг vino-svoe.ru (BeautifulSoup + Requests)
БД	PostgreSQL 15
Контейнеризация	Docker + Docker Compose
Деплой	Render (render.yaml, Docker env)

Содержание

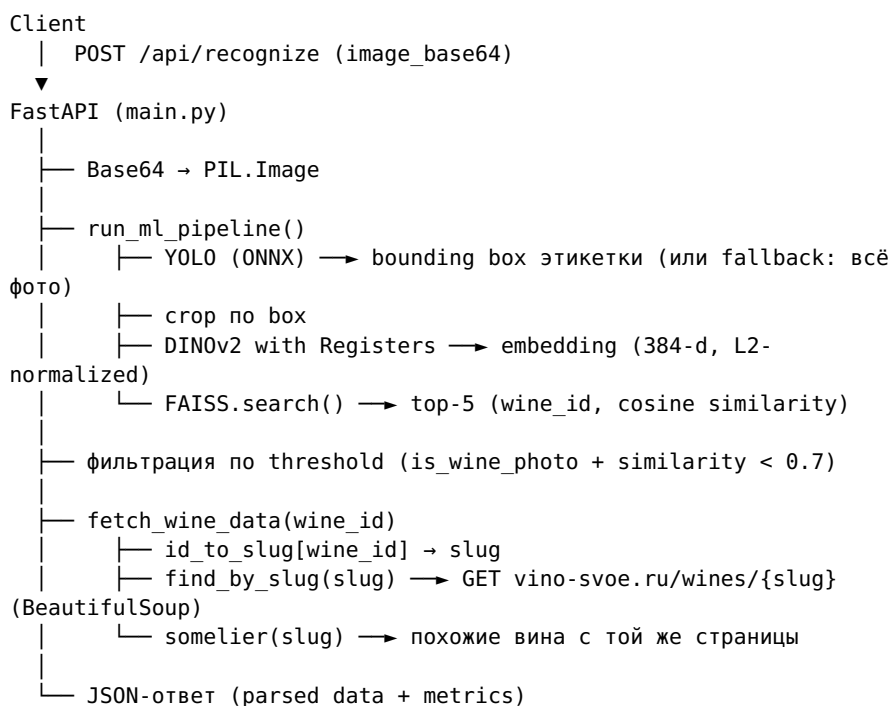
№	Раздел
1	Резюме архитектуры
2	Структура проекта
3	Стек и зависимости
4	Точка входа, инициализация и сборка
5	Сценарий распознавания вина

- 6 ML-пайплайн: YOLO → DINOv2 → FAISS
 - 7 Сетевой слой и API-контракт
 - 8 Парсинг карточки вина (vino-svoe.ru)
 - 9 Винная лента (/feed) и кэш
 - 10 Хранение данных: PostgreSQL и файловые артефакты
 - 11 Ошибки, фильтрация и устойчивость
 - 12 Конфигурация и переменные окружения
 - 13 Сборка, окружения и деплой
 - 14 Технический долг и рекомендации
 - 15 Onboarding нового backend-разработчика
 - 16 Справочник файлов и глоссарий
-

1. Резюме архитектуры

Backend представляет собой единый Python-сервис на FastAPI без разделения на отдельные модули/пакеты — вся бизнес-логика (ML-инференс, парсинг сайта, кэш ленты, HTTP-эндпоинты) находится в одном файле `main.py`. Это осознанно простая структура: сервис — это, по сути, ML-инференс-обёртка поверх стороннего сайта-каталога вин, а не самостоятельный источник истины о винах.

Ключевая схема запроса на распознавание:



Основные свойства решения

- Backend не хранит характеристики вин самостоятельно — они получаются “на лету” парсингом стороннего сайта `vino-svoe.ru` при каждом запросе к `/api/recognize` и `/api/memory`.
- PostgreSQL используется формально: таблица `wines` создаётся и наполняется `init.sql`, но в текущем коде `main.py` она нигде не читается и не пишется. Реальное сопоставление ID ↔ slug вина происходит через файл `wines_mapping.json`, а не через БД.
- FAISS-индекс (`wines_base.index`) — предвычисленная база `embeddings` вин, построенная офлайн; сам сервис в неё ничего

- не добавляет во время работы (read-only).
- Идентификация вина двухэтапная: сначала YOLO находит этикетку и обрезает фото (если уверенность выше порога), затем DINOv2 считает embedding для кропа (или всего фото, если этикетка не найдена), затем FAISS ищет ближайшие векторы по косинусной близости.
- Помимо основного similarity-порога используется дополнительный сигмоидальный F1-score (sigmoid_f1_score) для более интерпретируемой метрики уверенности, возвращаемой клиенту.
- Раздел “лента” (/feed) — независимый функционал: агрегирует новости с vino-svooe.ru и список вин из wines_mapping.json, кэшируя результат на 6 часов в памяти процесса.

Архитектурный вывод

Кодовая база — рабочий, но не строго слоистый ML-сервис: сетевые эндпоинты, ML-инференс, парсинг HTML и кэширование сосуществуют в одном файле на уровне модульных функций. Главная зона роста — вынесение слоёв (ML / scraping / caching / API) в отдельные модули, замена парсинга живого сайта на собственное хранилище данных о винах и устранение несогласованности между init.sql и фактически используемым wines_mapping.json.

2. Структура проекта

Путь	Назначение
main.py	Единственный модуль приложения: FastAPI app, ML-пайплайн, парсинг сайта, лента, все эндпоинты
requirements.txt	Python-зависимости (без torch/torchvision — они ставятся отдельной командой)
Dockerfile	Сборка образа backend на базе python:3.10-slim
docker-compose.yml	Локальный запуск web (backend) + db (PostgreSQL)
init.sql	DDL + INSERT для таблицы wines (используется PostgreSQL-контейнером при первом старте)
label_detector.onnx / label_detector.onnx.data	ONNX-модель YOLO для детекции этикетки (~24 МБ суммарно)
wines_base.index	FAISS-индекс (IndexFlatIP) embeddings вин, размерность 384
wines_mapping.json	Соответствие FAISS id → wine_slug, 1958 записей
render.yaml	Конфигурация деплоя на Render (Docker-сервис + managed PostgreSQL)

Рекомендуемая ментальная модель main.py (по секциям файла)

НАСТРОЙКИ / DEVICE

→ константы, порог CUDA/CPU

YOLO / DINOv2 / FAISS init	→ загрузка моделей при старте процесса
FAISS + MAPPING (lifespan)	→ асинхронная загрузка index +
wines_mapping.json	
REQUEST / FEED MODELS	→ pydantic-схемы запросов и ответов
FEED CACHE / SETTINGS	→ TTL-кэш винной ленты в памяти
DEBUG CROP	→ сохранение кропов на диск для
отладки	
YOLO LETTERBOX	→ препроцессинг изображения под YOLO
DINOv2 EMBEDDING	→ получение embedding
ФИЛЬТР / ML PIPELINE	→ run_ml_pipeline(),
sigmoid_f1_score()	
PARSE WINE SITE	→ find_by_slug(), somelier()
FETCH WINE / RESPONSE	→ fetch_wine_data(), parsed_info()
ЭНДПОИНТЫ	→ /feed, /api/recognize, /api/memory,
/api/test_for_slug	

3. Стек и зависимости

Технология	Версия / ограничение	Роль
Python	3.10 (slim образ)	Среда исполнения
FastAPI	из requirements.txt, без пиннинга	REST API framework
Uvicorn	без пиннинга	ASGI-сервер
PyTorch / torchvision	CPU build, устанавливается отдельно от requirements.txt	Инференс DINOv2
Transformers	без пиннинга	Загрузка facebook/dinov2- with-registers-small
onnxruntime	>=1.18.0	Инференс YOLO- модели детекции этикетки
faiss-cpu	без пиннинга	Векторный поиск похожих вин
opencv-python- headless	без пиннинга	Letterbox- препроцессинг изображения для YOLO
Pillow / pillow_heif	без пиннинга	Работа с изображениями, поддержка HEIC/HEIF
psycopg2-binary	без пиннинга	Драйвер PostgreSQL (подключён, но в текущем main.py не используется)
requests + beautifulsoup4	без пиннинга	Скрейпинг vino- svoe.ru
python-multipart	без пиннинга	Приём multipart/form-data в /api/test_for_slug
async-timeout	без пиннинга	Асинхронные таймауты

Принцип выбора зависимостей

Версии в `requirements.txt` в основном не зафиксированы (кроме `onnxruntime>=1.18.0`), что упрощает первую сборку, но не гарантирует воспроизводимость сборки во времени. PyTorch устанавливается отдельной командой (`--extra-index-url .../cpu`) до основного `pip install -r requirements.txt`, чтобы получить CPU-сборку, а не тянуть CUDA-зависимости по умолчанию.

4. Точка входа, инициализация и сборка

Порядок инициализации процесса

Инициализация тяжёлых ресурсов разделена на два этапа:

1. **На уровне модуля** (при импорте `main.py`, до создания `FastAPI` app): выбор device (CUDA/CPU), загрузка ONNX-сессии YOLO (`ort.InferenceSession`), загрузка `AutoImageProcessor` и `AutoModel` для DINOv2. Ошибка загрузки YOLO или DINOv2 приводит к `sys.exit(1)` — сервис не поднимается с нерабочими моделями.
2. **В `lifespan`-контексте `FastAPI`** (при старте ASGI-приложения): загрузка FAISS-индекса из `wines_base.index` и `wines_mapping.json` в глобальные переменные `index` и `id_to_slug`. Здесь же проверяется согласованность размерности индекса и `embedding`-модели, а также совпадение количества записей в `mapping` и числа векторов в индексе (несовпадение только логируется, не блокирует старт).

```
import main.py
├ device = cuda | cpu
├ YOLO ONNX session —(ошибка → sys.exit)
└ DINOv2 processor + model —(ошибка → sys.exit)

FastAPI(lifespan=lifespan)
└ lifespan startup
  ├── faiss.read_index(wines_base.index)
  ├── проверка index.d == embedding_dimension (иначе
RuntimeError)
  └ json.load(wines_mapping.json) → id_to_slug
```

Docker build

Параметр	Значение
Базовый образ	python:3.10-slim
Системные пакеты	build-essential, libpq-dev
PyTorch	CPU-сборка, --extra-index-url https://download.pytorch.org/whl/cpu
Точка входа	uvicorn main:app --host 0.0.0.0 -- port 8000
Порт	8000

Локальный запуск (Docker Compose)

`docker compose up --build` поднимает два сервиса: `db` (PostgreSQL 15, инициализация через `init.sql`) и `web` (backend, зависит от `db`). Volume `postgres_data` обеспечивает персистентность БД между перезапусками; `docker compose down -v` удаляет её вместе с контейнерами.

5. Сценарий распознавания вина

Полный пользовательский поток на стороне backend для POST /api/recognize:

1. Клиент отправляет { "image_base64": "<JPEG as Base64>" }
2. Base64 → PIL.Image (RGB)
3. run_ml_pipeline(image, orig_w, orig_h, 640, 640)
 - └ YOLO inference → best_score
 - └ если best_score > 0.25 → box_detected = True, кроп по этикетке
 - └ иначе → box_detected = False, используется всё фото
 - └ DINOv2 embedding по (кропу или полному фото)
 - └ FAISS.search(embedding, k=5) → top-5 (wine_id, similarity)
4. is_wine_photo(similarity, box_detected)
 - порог 0.45 (с box) или 0.55 (без box)
 - если similarity ниже порога: HTTP 404 "Вино на фото не распознано"
5. Дополнительный фильтр: top1 similarity < 0.7 → HTTP 404 "Точного совпадения в базе не найдено" (+ metrics в detail)
6. fetch_wine_data(wine_id)
 - └ id_to_slug[wine_id] → wine_slug
 - └ find_by_slug(wine_slug) → парсинг карточки вина с vino-svoe.ru
 - └ somelier(wine_slug) → парсинг блока рекомендаций с той же страницы
7. parsed_info(...) → JSON-ответ клиенту

Ответ /api/recognize

```
{
  "status": "success",
  "url": "https://vino-svoe.ru/wines/<slug>",
  "metrics": {
    "top1_f1": 0.0,
    "top5_scores": [...],
    "top5_candidates": [
      { "wine_id": 0, "slug": "...", "similarity": 0.0, "f1_score":
0.0 }
    ]
  },
  "parsed_data": {
    "name": "...", "description": "...", "factory": "...",
    "rate": "...", "area": "...", "sort": "...", "type": "...",
    "color": "...", "temperature": "...", "alcohol": "...",
    "dishes": [...], "wine_image": "data:image/jpeg;base64,...",
    "top5": [{"название", "data:image/webp;base64,...", "url"}, ...]
  }
}
```

Поле top5 в parsed_data — это не top-5 кандидатов FAISS, а результат somelier(): карточки “похожих вин”, спарсенные с блока рекомендаций на странице самого распознанного вина на vino-svoe.ru. Top-5 кандидатов FAISS отдельно доступны в metrics.top5_candidates.

6. ML-пайплайн: YOLO → DINOv2 → FAISS

YOLO (детекция этикетки)

Параметр	Значение
Модель	label_detector.onnx (ONNX Runtime, CPUExecutionProvider) 640 × 640, letterbox-

Входной размер	препроцессинг с паддингом 114,114,114
Формат выхода	[1, N, 5+], приводится к (8400, N) при необходимости транспонирования
Порог уверенности	YOLO_CONFIDENCE_THRESHOLD = 0.25
Поведение при низкой уверенности	Детекция не применяется, embedding считается по всему фото (box_detected = False)

Координаты box переводятся из модельного пространства (640×640 с паддингом) обратно в исходные пиксели фотографии через обратное масштабирование (scale) и вычитание паддинга (dw, dh), после чего клипуются в границы изображения.

DINOv2 with Registers (embedding)

Параметр	Значение
Модель	facebook/dinov2-with-registers-small
Размерность embedding	384 (hidden_size), должна совпадать с размерностью FAISS-индекса
Режимы агрегации	EMBEDDING_MODE=cls (по умолчанию, pooler_output) или EMBEDDING_MODE=mean (среднее по patch-токенам, за вычетом CLS-токена и register-токенов)
Нормализация	L2-нормализация (F.normalize) перед помещением в FAISS
Устройство	CUDA, если доступна, иначе CPU (автоопределение при старте)

FAISS (поиск похожих)

Используется IndexFlatIP (inner product) на L2-нормализованных векторах, что эквивалентно поиску по косинусной близости. Запрос возвращает top-5 кандидатов; лучший (top-1) используется как основной результат, весь top-5 — как метрики уверенности.

Двухуровневая фильтрация “это вино?”

1. **is_wine_photo()** — сравнивает cosine similarity top-1 с порогом, зависящим от того, была ли обнаружена этикетка: SIMILARITY_THRESHOLD_WITH_BOX = 0.45, SIMILARITY_THRESHOLD_NO_BOX = 0.55. Идея: если этикетка чётко обнаружена, требуется меньшая похожесть, чем при поиске по всему кадру.
2. **Жёсткий порог similarity < 0.7** — дополнительно отбрасывает совпадения, даже прошедшие первый фильтр, если top-1 similarity ниже 0.7. Оба фильтра приводят к HTTP 404, но с разными сообщениями.

sigmoid_f1_score

```
f1 = 1 / (1 + exp(-k * (similarity - s0))), s0 = 0.70, k = 15.0
```

Сигмоида превращает “сырую” косинусную похожесть в более интерпретируемую 0...1-метрику уверенности с резким переходом около порога $s_0 = 0.70$, которая и передаётся клиенту как f1_score.

7. Сетевой слой и API-контракт

Общие настройки FastAPI

CORS открыт полностью (`allow_origins=["*"]`, `allow_credentials=True`, все методы и заголовки) — ограничение источников на уровне приложения отсутствует.

Endpoint	Метод	Назначение
<code>/api/recognize</code>	POST	Распознать вино по Base64 JPEG (полный ML-пайплайн)
<code>/api/memory</code>	POST	Получить данные о вине по уже известному slug (без ML)
<code>/api/test_for_slug</code>	POST	Тестовый эндпоинт ML-пайплайна, возвращает только slug
<code>/feed</code>	GET	Кэшированная винная лента (новости + подборка вин)

POST `/api/recognize`

Запрос: `{ "image_base64": "<BASE64_IMAGE>" }`. Ответ — см. раздел 5. Ошибки: 400 при невалидном base64, 404 при непройденных ML-фильтрах, 500 при сбое пайплайна.

POST `/api/memory`

Запрос: `{ "memory_slug": "wine-slug" }`. Не выполняет ML-инференс — сразу вызывает `memory_slug()`, который повторяет `find_by_slug()` + `somelier()` для уже известного slug. Ответ имеет тот же формат, что и `/api/recognize`, но без блока `metrics` (передаётся `None`).

POST `/api/test_for_slug`

Принимает изображение тремя способами: как `UploadFile` (multipart), как `image_base64` (form-данные) или как сырое тело запроса (JSON с полем `image_base64/image`, либо raw body). Выполняет тот же ML-пайплайн, что и `/api/recognize`, но в ответ отдаёт только `{ "slug": "..."` } — используется для проверки корректности распознавания без парсинга карточки вина.

GET `/feed`

Синхронный эндпоинт без тела запроса; логика полностью строится вокруг in-memory кэша с TTL (раздел 9). Возвращает `FeedResponse` (см. модели ниже) с заголовками `Cache-Control` и, при отдаче устаревшего кэша, `X-Feed-Cache: stale`.

Pydantic-модели запросов/ответов

Модель	Поля
--------	------

ImageRequest	image_base64: str
WineSlug	memory_slug: str
ImageTest	test_slug: str (объявлена, но эндпоинт /api/test_for_slug фактически не использует эту модель — см. раздел 14)
FeedNewsItem	title, description, url, image_url, source, published_at
FeedWineItem	slug, name, image_url
FeedPayload	news: list[FeedNewsItem], wines: list[FeedWineItem]
FeedResponse	status: str, feed: FeedPayload

8. Парсинг карточки вина (vino-svoe.ru)

Backend не хранит характеристики вин — при каждом запросе к /api/recognize, /api/memory и /feed (для блока wines) он обращается напрямую к <https://vino-svoe.ru>.

find_by_slug(wine_slug)

Загружает <https://vino-svoe.ru/wines/{slug}> и извлекает через BeautifulSoup по CSS-классам: описание (wine-page__description), название (wine-main-title-block__title), производителя (wine-main-title-block__manufacturer), рейтинг (wine-main-title-block__rating-text), список атрибутов (wine-detail-info__detail-value → регион/сорт/тип/цвет), список числовых характеристик (wine-hero-block__card-value → температура/крепость), список блюд (wine-dish-item__name) и изображение бутылки (wine-hero-block__bottle), которое скачивается отдельным запросом и кодируется в Base64 прямо в ответе API.

somelier(wine_slug)

Парсит блок карточек-рекомендаций (a.wine-item, до 5 штук) на той же странице вина: название, ссылку на рекомендованное вино и его изображение (скачивается и кодируется в Base64 как data:image/webp;base64,...). При отсутствии рекомендаций возвращает список из 5 заглушек "Нет информации".

Устойчивость парсинга

Оба парсера полностью полагаются на текущую HTML/CSS-разметку vino-svoe.ru: изменение имён классов на сайте-источнике сломает извлечение данных без изменений на стороне backend. При сетевых ошибках или ошибках парсинга функции не бросают исключение наружу, а подставляют текстовые заглушки ("Не удалось загрузить информацию", "Ошибка подключения к сайту") во все поля результата, чтобы /api/recognize не падал полностью из-за недоступности стороннего сайта.

9. Винная лента (/feed) и кэш

Источники данных ленты

- **Новости** — HTML-страницы 7 фиксированных подразделов vino-

svoe.ru/category/articles/{vino,story,news,sobytiya,koktejli,intervyu,instrumenty}. Из каждой категории собираются ссылки на статьи, затем каждая статья параллельно загружается и парсится (заголовок, описание, изображение, дата публикации).

- **Вина** — берутся напрямую из `wines_mapping.json` (`id_to_slug`), без обращения к FAISS; для каждого `slug` лёгкой версией `find_by_slug()` (`_fetch_feed_wine`) подгружаются только название и изображение бутылки (без описания и без рекомендаций).

Кэш

<code>_feed_cache_bytes</code>	— уже сериализованный JSON-ответ (bytes)
<code>_feed_cache_timestamp</code> (event loop time)	— время последнего успешного обновления
<code>_feed_cache_lock</code>	— <code>asyncio.Lock</code> , защита от параллельного обновления
<code>FEED_CACHE_TTL</code>	— 6 часов

Логика GET /feed: при валидном кэше (моложе TTL) — мгновенный ответ из памяти. При невалидном — захват `_feed_cache_lock`, повторная проверка кэша (double-checked locking), затем блокирующая сборка нового payload в отдельном потоке (`asyncio.to_thread`), чтобы не блокировать event loop. При ошибке сборки нового кэша, если есть предыдущий валидный (пусть и устаревший) кэш — отдаётся он с пометкой X-Feed-Cache: stale; если кэша ещё не было вообще — клиент получает 500.

Отбор и сортировка новостей

Из всех спарсенных статей сначала гарантированно берётся по одной самой свежей статье из **каждого** из 7 подразделов (чтобы лента не была перекошена в одну тематику), оставшиеся места (до `FEED_NEWS_LIMIT = 20`) заполняются самыми свежими статьями глобально по дате публикации, после чего итоговый список ещё раз сортируется по дате целиком. Статья без распознанной даты публикации в ленту не попадает — иначе её невозможно корректно отсортировать.

Лимиты и таймауты

Параметр	Значение
<code>FEED_NEWS_LIMIT</code>	20
<code>FEED_WINE_LIMIT</code>	30
<code>FEED_ARTICLES_PER_CATEGORY</code>	8 (на подраздел)
<code>FEED_CATEGORY_TIMEOUT</code>	8 сек
<code>FEED_ARTICLE_TIMEOUT</code>	8 сек
<code>FEED_WINE_TIMEOUT</code>	5 сек

10. Хранение данных: PostgreSQL и файловые артефакты

PostgreSQL

`docker-compose.yml` поднимает `postgres:15-alpine` с БД `wine_db`, пользователем `wine_user` и инициализацией из `init.sql`, который создаёт таблицу `wines(id INT PRIMARY KEY, wine_slug VARCHAR(255))` и наполняет её ~1221 записью.

Важное расхождение: в текущей версии main.py эта таблица нигде не читается через psycopg2 — драйвер присутствует в зависимостях, но соответствующий код подключения к БД в файле отсутствует. Фактическое сопоставление id → slug, которым пользуется весь ML-пайплайн, полностью происходит через wines_mapping.json (1958 записей — больше, чем в init.sql). PostgreSQL на данный момент — задел на будущее хранение (например, для миграции с live-парсинга сайта на собственную БД вин), а не используемый в рантайме компонент.

Файловые ML-артефакты

Файл	Назначение	Требование согласованности
label_detector.onnx + .onnx.data	Весы YOLO-детектора этикетки	Формат выхода должен соответствовать run_ml_pipeline() (координаты в формате xc,yc,w,h + confidence)
wines_base.index	FAISS IndexFlatIP, векторы 384-d	Размерность должна совпадать с embedding_dimension DINOv2 (проверяется при старте, иначе RuntimeError)
wines_mapping.json	{ "faiss_id": "wine_slug" }	Количество записей должно совпадать с index.ntotal (при расхождении — только предупреждение в лог)

11. Ошибки, фильтрация и устойчивость

Распознавание (/api/recognize, /api/test_for_slug)

Событие	HTTP-статус	Реакция
Невалидный Base64 / не открывается как изображение	400	"Не удалось прочитать base64 строку: ..."
similarity ниже адаптивного порога (is_wine_photo)	404	"Вино на фото не распознано"
similarity top-1 ниже 0.7 (даже после прохождения первого фильтра)	404	{"message": "Точного совпадения в базе не найдено", "metrics": ...}
Любая другая ошибка в ML/сетевом пайплайне	500	Текст исключения в detail, полный traceback в лог

/feed

Событие	HTTP-статус	Реакция
Кэш валиден	200	Мгновенный ответ из памяти
Кэш невалиден, сборка успешна	200	Новый кэш, Cache-Control: max-age=21600
Кэш невалиден, сборка упала, есть старый кэш	200	Старый кэш + заголовок X-Feed-Cache: stale
Кэш невалиден, сборка упала, кэша никогда не было	500	"Не удалось сформировать винную ленту: ..."

Устойчивость к недоступности vino-svcoe.ru

Парсинг карточки вина (find_by_slug) и рекомендаций (somelier) не пробрасывают сетевые исключения наружу — при ошибке подставляются текстовые заглушки, и /api/recognize/api/memory всё равно возвращают 200 с частично заполненными (или полностью заглушечными) данными о вине. Это означает, что клиент не получает явного сигнала о недоступности стороннего сайта — только текст "Ошибка подключения к сайту" внутри обычно валидных полей ответа.

12. Конфигурация и переменные окружения

Переменная	По умолчанию	Назначение
SIMILARITY_THRESHOLD_WITH_BOX	0.45	Порог cosine similarity при обнаруженной этикетке
SIMILARITY_THRESHOLD_NO_BOX	0.55	Порог cosine similarity, если этикетка не обнаружена YOLO
EMBEDDING_MODE	cls	Способ агрегации DINOv2-embedding: cls (pooler_output) или mean (среднее по patch-токенам)

Прочие параметры (пороги YOLO, размер входа, лимиты и таймауты ленты, TTL кэша) защиты константами в начале main.py и не вынесены в переменные окружения.

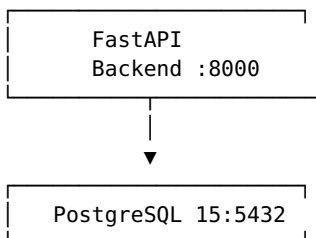
Переменные окружения PostgreSQL (Docker Compose / Render)

Переменная	Значение (локально)	Назначение
POSTGRES_DB	wine_db	Имя БД
POSTGRES_USER	wine_user	Пользователь

На Render (`render.yaml`) эти же переменные генерируются динамически из managed-БД (`fromDatabase`), а не задаются напрямую.

13. Сборка, окружения и деплой

Локально (Docker Compose) — рекомендуемый способ



`docker compose up --build` собирает образ backend, поднимает PostgreSQL, выполняет `init.sql`, запускает Uvicorn на порту 8000. Готовность подтверждается логами [YOLO] Модель загружена, [DINOv2] Модель успешно загружена, [FAISS] База успешно загружена. и доступностью Swagger UI (`/docs`) / ReDoc (`/redoc`).

Локально без Docker

Требуется Python 3.10, ручная установка CPU-версии PyTorch до `requirements.txt`, самостоятельный запуск PostgreSQL и `uvicorn main:app --host 0.0.0.0 --port 8000`.

Деплой на Render (`render.yaml`)

Render-сервис `wine-backend` собирается из `Dockerfile` (`env: docker`), managed PostgreSQL создаётся отдельным `databases`-ресурсом; учётные данные БД (`POSTGRES_PASSWORD`, `POSTGRES_HOST`) прокидываются в сервис динамически через `fromDatabase`. Закомментированный блок `DATABASE_URL` — задел на переход к единой DSN-строке подключения, которая в текущем `main.py` всё равно не используется (раздел 10).

14. Технический долг и рекомендации

- **`process_and_convert_to_base64` не определена.** Эндпоинт `/api/test_for_slug` вызывает эту функцию в самом начале тела, но нигде в `main.py` она не объявлена и не импортирована — вызов эндпоинта с `file` или `image_base64` в текущем виде завершится `NameError`, обернутым в `HTTPException(400)`. Ниже по коду того же эндпоинта есть отдельный, независимо работающий блок декодирования через `data.test_slug` (модель `ImageTest`), который на сам `data` не имеет доступа (параметр запроса называется иначе) — эндпоинт фактически нерабочий в текущем состоянии и требует рефакторинга.
- **PostgreSQL не используется в рантайме.** `psycopg2-binary` установлен, `init.sql` наполняет таблицу `wines`, но код подключения и чтения из БД в `main.py` отсутствует; источником истины *de facto* является `wines_mapping.json`. Стоит либо реализовать реальное чтение из БД, либо убрать

неиспользуемую инфраструктуру, чтобы не вводить в заблуждение.

- **Расхождение количества вин.** `init.sql` содержит ~1221 запись, `wines_mapping.json` — 1958. Это разные, не синхронизированные между собой источники данных об одном и том же домене.
- **Полностью открытый CORS** (`allow_origins=["*"]`) без ограничения по домену — приемлемо для разработки, но требует пересмотра перед продакшен-релизом с публичным API.
- **Жёсткая зависимость от вёрстки `vino-svoe.ru`.** Все данные о винах (название, описание, рейтинг, изображение, рекомендации) извлекаются по CSS-классам стороннего сайта без версионирования и без fallback-стратегии на случай редизайна сайта-источника.
- **DEBUG_SAVE_CROPS = True по умолчанию.** В продакшен-конфигурации это приведёт к постоянному накоплению файлов в `debug_crops/` и выводу полного base64 кропа в логи (чанками), что раздувает логи и создаёт риск утечки пользовательских фото через лог-агрегатор.
- **Отсутствие версионирования модели/индекса.** Нет проверки версии ONNX-модели/FAISS-индекса в ответе API — при обновлении `wines_base.index` невозможно на лету понять, на какой версии базы построен конкретный ответ.
- **Нет ретраев и `circuit breaker` для запросов к `vino-svoe.ru`.** Единичный `requests.get` с фиксированным коротким таймаутом (5–8 сек) без повторов — при деградации стороннего сайта ответы будут массово превращаться в заглушки "Нет информации".

15. Onboarding нового backend-разработчика

1. Изучить `main.py` сверху вниз по секциям (раздел 2) — файл линейный, специальной архитектуры пакетов нет.
2. Поднять сервис локально через `docker compose up --build`, проверить `/docs` и логи загрузки моделей.
3. Прогнать `/api/recognize` с тестовым фото вина через Swagger UI, изучить структуру ответа и `metrics`.
4. Обратить внимание на раздел 14 (технический долг) перед внесением изменений в `/api/test_for_slug` и в код, связанный с PostgreSQL.
5. При изменении `label_detector.onnx` или `wines_base.index` — обязательно синхронизировать размерность `embedding` и содержимое `wines_mapping.json`, иначе сервис не запустится (`RuntimeError`) или начнёт указывать не на те вина.

16. Справочник файлов и глоссарий

Файл	Ключевая роль
<code>main.py</code>	Весь backend: FastAPI app, ML-пайплайн, скрейпинг, кэш ленты, эндпоинты
<code>label_detector.onnx(.data)</code>	Весы YOLO-модели детекции этикетки
<code>wines_base.index</code>	FAISS-индекс <code>embeddings</code> вин (384-d, <code>IndexFlatIP</code>)
<code>wines_mapping.json</code>	Реальное сопоставление FAISS <code>id</code> → <code>wine_slug</code>

init.sql	DDL/данные для PostgreSQL-таблицы wines (в рантайме не читается)
Dockerfile / docker-compose.yml	Сборка и локальный запуск backend + PostgreSQL
render.yaml	Конфигурация деплоя на Render

Глоссарий

Термин	Значение
YOLO	Object-detection модель, здесь — детектор этикетки на фото бутылки
ONNX Runtime	Runtime для выполнения экспортированных в ONNX моделей (используется для YOLO)
DINOv2 with Registers	Self-supervised vision-модель Meta, используется как экстрактор embeddings
Embedding	Векторное представление изображения (384-d), используется для сравнения похожести
FAISS	Библиотека Facebook AI для эффективного поиска ближайших векторов
IndexFlatIP	Тип индекса FAISS, поиск по скалярному произведению (на нормализованных векторах = cosine similarity)
Cosine similarity	Косинусная мера близости двух векторов, диапазон [-1, 1]
slug	Человекочитаемый URL-идентификатор вина на vino-svoe.ru, ключ сопоставления с FAISS id
Letterbox	Способ ресайза изображения с сохранением пропорций и паддингом до квадратного входа модели
lifespan	Механизм FastAPI для асинхронной инициализации/остановки ресурсов приложения
TTL-кэш	Кэш с ограниченным временем жизни записи (здесь — 6 часов для /feed)